

Lab 1

Objective: To develop a console application in C# that takes user input for their name and age, and prints a welcome message.

Theory

C# is a modern, object-oriented programming language developed by Microsoft and widely used for building applications on the .NET platform. A console application is a basic form of .NET application that runs in the command line and interacts with users through text.

This lab focuses on the use of:

- `Console.ReadLine()` to receive input from the user.
- `Convert.ToInt32()` to convert string input to integer.
- `Console.WriteLine()` and string interpolation to display output.

Understanding how to handle input and output in a console environment is essential for building interactive applications. These fundamentals form the basis for advanced topics like file handling, database access, and GUI development in .NET.

Code

```
using System;

class Program
{
    static void Main()
    {
        Console.Write("Enter your name: ");
        string name = Console.ReadLine();

        Console.Write("Enter your age: ");
        string ageInput = Console.ReadLine();
        int age = Convert.ToInt32(ageInput);
        Console.WriteLine($"Welcome, {name}! You are {age} years old.");
    }
}
```

Output

```
Enter your name: deep  
Enter your age: 11  
Welcome, deep! You are 11 years old.  
  
=== Code Execution Successful ===
```

Discussion

The code demonstrates how to create a simple C# console application that interacts with the user. The program takes string input for the name and age, converts the age to an integer, and displays a formatted welcome message using string interpolation.

Conclusion

The console application successfully captures user input and displays a customized welcome message. This lab helped in understanding essential input/output operations in C#, providing a strong foundation for building more complex .NET applications.

Lab 2

Objectives: To create a C# program that demonstrates valid and invalid identifiers and shows the use of various C# keywords

Theory

In C#, identifiers are the names used to identify variables, methods, classes, etc. Valid identifiers must follow specific rules:

- Can contain letters, digits, and underscores.
- Must not begin with a digit.
- Cannot be the same as reserved keywords
- Are case-sensitive.

C# also provides a rich set of keywords that serve specific purposes in the language, such as data types (int, string), control structures (if, else), and modifiers (public, private, static).

Understanding valid naming and how to correctly use keywords is critical in writing readable and syntactically correct C# code.

Code

```
using System;

class IdentifierDemo
{
    static void Main(string[] args)
    {
        // Valid identifiers

        int age = 11;           // Simple variable

        string _name = "deep"; // Starts with an underscore

        string @class = "BCA5"; // Reserved keyword with @ prefix


        // Invalid identifiers

        // int 123number = 10; // Starts with a number

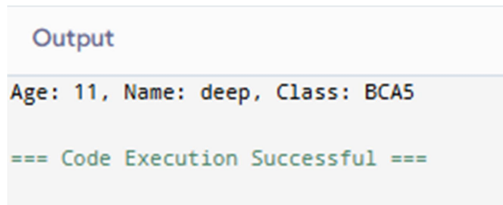
        // string void = "Invalid"; // Reserved keyword without @

        // string user name = "Invalid"; // Contains spaces
    }
}
```

```
// Output valid identifiers

        Console.WriteLine($"Age:    {age},    Name:    {_name},    Class:
{@class}");
    }
}
```

Output



The screenshot shows a code execution environment. At the top, the word "Output" is displayed in a blue font. Below it, the program's output is shown in a monospaced font: "Age: 11, Name: deep, Class: BCAS". At the bottom, a green message indicates "=== Code Execution Successful ===".

Discussion

This program demonstrates:

- The rules for naming identifiers in C#.
- How reserved keywords can be used as identifiers with the @ prefix.
- Examples of common errors when defining identifiers.

Errors from invalid identifiers: If invalid identifier lines are uncommented following compilation errors occurs:

1. Identifiers cannot start with a number.
2. 'void' is a reserved keyword.
3. Invalid syntax due to spaces in identifier names.

Conclusion

The program successfully demonstrates valid and invalid identifiers and shows how reserved keywords can be used as identifiers. By understanding and adhering to syntax rules, developers can avoid common pitfalls and write cleaner, error-free code.

Lab 3

Objective: To modify a C# program by adding appropriate single-line and multi-line comments, making the code more understandable and maintainable.

Theory

Comments in C# are non-executable lines that describe or explain code. They are useful for documentation, debugging, and collaboration.

- **Single-line comments** use `//` and are placed before or next to code statements.
- **Multi-line comments** use `/* */` and are used for larger descriptions or block comments.

Commenting is an essential practice for writing readable and professional code, especially in collaborative or long-term projects.

Code

```
using System;

class CommentedProgram
{
    static void Main()
    {
        /*
            This program collects a user's name and age,
            then prints a personalized welcome message.
        */

        // Ask for the user's name
        Console.Write("Enter your name: ");

        string name = Console.ReadLine(); // Store user input into
'name'

        // Ask for the user's age
        Console.Write("Enter your age: ");
```

```
        string ageInput = Console.ReadLine(); // Read age as a
string

        // Convert age to integer for further processing

        int age = Convert.ToInt32(ageInput);

        // Display a welcome message

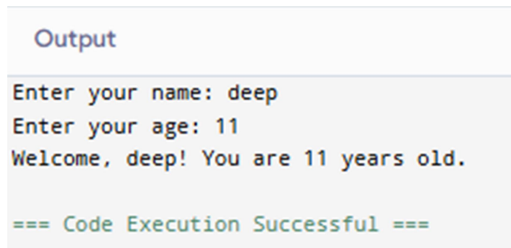
        Console.WriteLine($"Welcome, {name}! You are {age} years
old.");

        // End of the program

    }

}
```

Output



The screenshot shows a code execution environment with the following output:

```
Output
Enter your name: deep
Enter your age: 11
Welcome, deep! You are 11 years old.

=== Code Execution Successful ===
```

Discussion

This program demonstrates how to add both single-line and multi-line comments in C#. Each part of the code is explained, making it easier to understand the purpose of every section. Such comments are useful not only for others reading the code but also for the original programmer when revisiting the code after some time.

Conclusion

The modified program now includes explanatory comments that improve code readability and understanding. Adding comments is a best practice in programming, and this lab helps students develop good documentation habits early in their coding journey.

Lab 4

Objective: To develop a basic calculator in C# that demonstrates the use of different data types and various arithmetic and logical operators.

Theory

C# supports several numerical data types, including:

- int – for whole numbers.
- float – for single-precision floating-point numbers.
- double – for double-precision floating-point numbers.

Arithmetic operators include +, -, *, /, % and are used to perform mathematical operations.

Logical operators like &&, ||, and ! are used in boolean expressions for decision making.

This lab demonstrates how to work with different data types and operators in one program to perform basic calculator operations.

Code

```
using System;

class Program
{
    static void ArithmeticCalculator()
    {
        int intNum1 = 5, intNum2 = 3;

        float floatNum1 = 7.5f, floatNum2 = 2.2f;

        double doubleNum1 = 12.345, doubleNum2 = 4.567;

        Console.WriteLine("Arithmetic Operations:");

        // Integer arithmetic

        Console.WriteLine($"Int: {intNum1} + {intNum2} = {intNum1 + intNum2}");

        Console.WriteLine($"Int: {intNum1} - {intNum2} = {intNum1 - intNum2}");

        // Float arithmetic
```

```

        Console.WriteLine($"Float: {floatNum1} * {floatNum2} =
{floatNum1 * floatNum2}");

        Console.WriteLine($"Float: {floatNum1} / {floatNum2} =
{floatNum1 / floatNum2}");

        // Double arithmetic

        Console.WriteLine($"Double: {doubleNum1} + {doubleNum2} =
{doubleNum1 + doubleNum2}");

        Console.WriteLine($"Double: {doubleNum1} - {doubleNum2} =
{doubleNum1 - doubleNum2}");

    }

    static void LogicalCalculator()
    {
        int val1 = 1, val2 = 0;

        Console.WriteLine("\nLogical Operations:");

        Console.WriteLine($"Logical AND: {val1} && {val2} =
{(val1 != 0 && val2 != 0)}");

        Console.WriteLine($"Logical OR: {val1} || {val2} = {(val1 !=
0 || val2 != 0)}");

        Console.WriteLine($"Logical NOT: !{val1} = {!(val1 != 0)}");

    }

    static void Main()
    {
        ArithmeticCalculator();

        LogicalCalculator();

    }
}

```


Output

```
Arithmetic Operations:
Int: 5 + 3 = 8
Int: 5 - 3 = 2
Float: 7.5 * 2.2 = 16.5
Float: 7.5 / 2.2 = 3.409091
Double: 12.345 + 4.567 = 16.912
Double: 12.345 - 4.567 = 7.778

Logical Operations:
Logical AND: 1 && 0 = False
Logical OR: 1 || 0 = True
Logical NOT: !1 = False
```

Discussion

This program performs calculations using three different numeric types (int, float, double) and demonstrates the use of arithmetic operators like +, /, *, %. It also shows how logical operators (&&, !, ==, >) work in C# to evaluate boolean expressions.

By mixing data types and operators, the program reinforces how operations behave differently depending on the context and type used.

Conclusion

The calculator program effectively showcases multiple data types and how they interact with arithmetic and logical operators. This lab builds a practical foundation for writing mathematical and decision-based logic in more complex .NET applications.

Lab 5

Objective: To develop a C# program that performs basic string manipulations

Theory

In C#, the string class provides built-in methods and properties for text manipulation. String operations are essential in many applications like form processing, file reading, and data formatting.

This lab demonstrates three common string operations:

- **Reversing a string:** By converting the string to a character array and reversing it.
- **Counting vowels:** Using a loop and checking each character.
- **Converting to uppercase:** Using the ToUpper() method of the string class.

Mastering these operations helps in handling and processing textual data effectively.

Code

```
using System;

class StringManipulation
{
    static void Main()
    {
        // Input string
        Console.Write("Enter a string: ");

        string input = Console.ReadLine();

        // Reverse the string
        char[] charArray = input.ToCharArray();
        Array.Reverse(charArray);

        string reversed = new string(charArray);

        Console.WriteLine("Reversed String: " + reversed);
    }
}
```

```

        // Count vowels

        int vowelCount = 0;

        foreach (char c in input.ToLower())
        {
            if ("aeiou".Contains(c))
            {
                vowelCount++;
            }
        }

        Console.WriteLine("Number of Vowels: " + vowelCount);

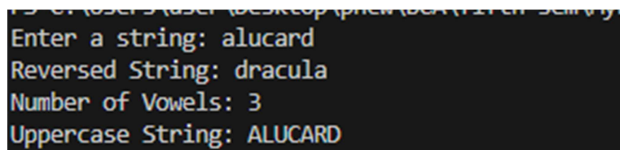
        // Convert to uppercase

        string upperCase = input.ToUpper();

        Console.WriteLine("Uppercase String: " + upperCase);
    }
}

```

Output



```

C:\Users\User\Desktop\new\bin\Debug>dotnet run
Enter a string: alucard
Reversed String: dracula
Number of Vowels: 3
Uppercase String: ALUCARD

```

Discussion

The program demonstrates key string manipulation tasks:

- Reversing is done by converting the string to a character array and using `Array.Reverse()`.
- Vowel counting uses a loop and `Contains()` with lowercase conversion for case-insensitivity.
- Uppercasing is handled by the `ToUpper()` method.

These are fundamental string operations that form the basis for more advanced tasks like validation, formatting, and parsing.

Conclusion

This lab effectively shows how to manipulate strings using built-in C# methods. It reinforces the importance of string operations in everyday programming tasks and builds confidence in handling textual data.

Lab 6

Objective: To create a C# program that takes 5 integer inputs from the user, stores them in an array, and calculates the total sum and average

Theory

In C#, an array is a fixed-size collection of elements of the same type. Arrays are useful for storing and processing multiple values together.

This lab also demonstrates:

- Accepting input using `Console.ReadLine()` and converting to `int`.
- Calculating **sum** using a loop.
- Computing **average** by dividing the sum by the number of elements.

These operations build essential skills in array handling and numeric processing.

Code

```
using System;

class ArraySumAverage
{
    static void Main()
    {
        int[] numbers = new int[5]; // Array to store 5 integers

        int sum = 0;

        Console.WriteLine("Enter 5 integers:");

        for (int i = 0; i < 5; i++)
        {
            Console.Write($"Number {i + 1}: ");

            numbers[i] = Convert.ToInt32(Console.ReadLine());

            sum += numbers[i];
        }

        double average = (double)sum / numbers.Length;
```

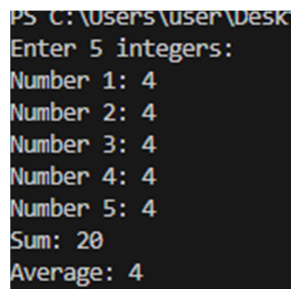
```
        Console.WriteLine("Sum: " + sum);

        Console.WriteLine("Average: " + average);

    }

}
```

Output



```
PS C:\Users\user\Desktop>
Enter 5 integers:
Number 1: 4
Number 2: 4
Number 3: 4
Number 4: 4
Number 5: 4
Sum: 20
Average: 4
```

Discussion

This program collects user input, stores it in an array, and uses a loop to calculate the **sum**. The average is computed using typecasting to ensure decimal division.

This reinforces how arrays and loops can be combined to solve basic data processing problems, such as calculating totals and averages.

Conclusion

The program successfully demonstrates array usage in C#, user input collection, and numeric operations. Understanding how to work with arrays and calculate statistical values is vital in many real-world programming scenarios.

Lab 7

Objective: To write a C# program that demonstrates the difference between passing parameters by value and by reference using methods

Theory

In C#, method parameters can be passed:

- By value (default): A copy of the variable is passed. Changes made inside the method do not affect the original variable.
- By reference (ref keyword): The method works with the original variable. Changes made inside the method affect the original variable.

This concept is important for memory management and understanding how variables behave across method calls.

Code

```
using System;

class ValueVsReference
{
    // Method that takes parameter by value
    static void PassByValue(int number)
    {
        number = number + 10;

        Console.WriteLine("Inside PassByValue: " + number);
    }

    // Method that takes parameter by reference
    static void PassByReference(ref int number)
    {
        number = number + 10;

        Console.WriteLine("Inside PassByReference: " + number);
    }
}
```

```

static void Main()
{
    int a = 5;

    int b = 5;

    Console.WriteLine("Original value of a (by value): " + a);

    PassByValue(a);

    Console.WriteLine("After PassByValue: " + a);

    Console.WriteLine();

    Console.WriteLine("Original value of b (by reference): " +
b);

    PassByReference(ref b);

    Console.WriteLine("After PassByReference: " + b);

}
}

```

Output

```

Original value of a (by value): 5
Inside PassByValue: 15
After PassByValue: 5

Original value of b (by reference): 5
Inside PassByReference: 15
After PassByReference: 15

```

Discussion

- In PassByValue(), the original variable a remains unchanged because the method only worked on a copy.
- In PassByReference(), the original variable b is updated since the method operated on the original memory location.

This difference is essential when deciding how to manipulate data within functions, especially with large objects or when you want a method to modify original data.

Conclusion

The program clearly demonstrates how passing by value and by reference behaves differently in C#. Understanding this concept helps in designing methods that either preserve or modify input data intentionally.

Lab 8

Objective: To create a C# program that determines and prints a student's grade based on their marks using both if-else and switch-case statements

Theory

Control flow statements like if-else and switch-case are used to make decisions based on conditions:

- if-else is best for range-based decisions.
- switch-case is suitable for discrete values.

This program uses if-else to determine the grade range and a switch-case to display a message based on the grade.

Code

```
using System;

class StudentGrade
{
    static void Main()
    {
        Console.Write("Enter marks (0-100): ");

        int marks = Convert.ToInt32(Console.ReadLine());

        string grade;

        if (marks >= 90 && marks <= 100)
            grade = "A";

        else if (marks >= 80)
            grade = "B";

        else if (marks >= 70)
            grade = "C";

        else if (marks >= 60)
            grade = "D";

        else if (marks >= 0)
```

```
        grade = "F";

else

    grade = "Invalid";

Console.WriteLine("Grade: " + grade);

switch (grade)
{
    case "A":

        Console.WriteLine("Excellent!");

        break;

    case "B":

        Console.WriteLine("Very Good!");

        break;

    case "C":

        Console.WriteLine("Good.");

        break;

    case "D":

        Console.WriteLine("Needs Improvement.");

        break;

    case "F":

        Console.WriteLine("Fail. Try harder next time.");

        break;

    default:

        Console.WriteLine("Invalid input.");

        break;

}

}

}
```

Output

```
Enter marks (0-100): 86
Grade: B
Very Good!
PS C:\Users\user\Desktop> gcc.c
```

Discussion

The program uses if-else to categorize marks into grade letters (A–F). Then, a switch-case statement is used to print a corresponding message based on the grade. This approach shows how to combine both decision-making constructs effectively in a real-world scenario.

Conclusion

This lab demonstrates how to use if-else for complex conditions and switch-case for specific outcomes. These control structures are fundamental in making programs behave dynamically based on input.

Lab 9

Objective: To write a C# program that generates the Fibonacci series up to N terms using all three types of loops: for, while, and do-while

Theory

The Fibonacci series is a sequence where each term is the sum of the two preceding ones, starting from 0 and 1.

Example: 0, 1, 1, 2, 3, 5, 8...

This lab shows how different loops (for, while, do-while) can be used to perform the same logic with slight variations in structure and flow control.

Code

```
using System;

class FibonacciDemo
{
    static void Main()
    {
        Console.WriteLine("Enter the number of terms (N): ");
        int n = Convert.ToInt32(Console.ReadLine());

        Console.WriteLine("\nUsing for loop:");
        int a = 0, b = 1, c;
        for (int i = 1; i <= n; i++)
        {
            Console.Write(a + " ");

            c = a + b;
            a = b;
            b = c;
        }
    }
}
```

```
Console.WriteLine("\n\nUsing while loop:");

a = 0; b = 1;

int count = 1;

while (count <= n)

{

    Console.Write(a + " ");

    c = a + b;

    a = b;

    b = c;

    count++;

}


Console.WriteLine("\n\nUsing do-while loop:");

a = 0; b = 1;

count = 1;

if (n > 0)

{

    do

    {

        Console.Write(a + " ");

        c = a + b;

        a = b;

        b = c;

        count++;

    } while (count <= n);

}
```

```
}  
  
}
```

Output

```
Enter the number of terms (N): 3  
  
Using for loop:  
0 1 1  
  
Using while loop:  
0 1 1  
  
Using do-while loop:  
0 1 1
```

Discussion

The program calculates the Fibonacci series using:

- A for loop for fixed iterations.
- A while loop with manual incrementing.
- A do-while loop which guarantees at least one iteration.

Each loop correctly produces the same output, showing how different control structures can be used interchangeably depending on the situation.

Conclusion

This lab successfully demonstrates generating the Fibonacci sequence using all three fundamental loop types. Understanding these loop structures is essential for iterative tasks in programming.

Lab 10

Objective: To create a user-defined namespace PNC and define a Student class inside it with fields for Name, Gender, and RollNo, including a constructor to initialize these fields.

Theory

Namespaces in C# are used to organize code and avoid name conflicts. A user-defined namespace helps in logically grouping related classes. A class is a user-defined data type in C# which contains fields, methods, constructors, etc. A constructor is a special method used to initialize class members when an object is created

Code

```
using System;

namespace PNC
{
    public class Student
    {
        public string Name { get; set; }

        public string Gender { get; set; }

        public int RollNo { get; set; }

        public Student(string name, string gender, int rollNo)
        {
            Name = name;

            Gender = gender;

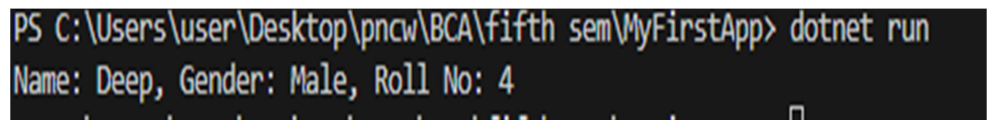
            RollNo = rollNo;
        }

        public void DisplayInfo()
        {
            Console.WriteLine($"Name: {Name}, Gender: {Gender}, Roll No: {RollNo}");
        }
    }
}
```



```
}  
  
}  
  
}  
  
class Program  
{  
  
    static void Main()  
  
    {  
  
        var student = new PNC.Student("Deep", "Male", 004);  
  
        student.DisplayInfo();  
  
    }  
  
}
```

Output



```
PS C:\Users\user\Desktop\pncw\BCA\fifth sem\MyFirstApp> dotnet run  
Name: Deep, Gender: Male, Roll No: 4
```

Conclusion

The objective of creating a user-defined namespace PNC and defining a Student class with a constructor was successfully achieved. The program ran correctly and displayed the student information using the object of the class

Lab 11

Objective: To add a destructor to the Student class that outputs a message when an instance is garbage-collected.

Theory

A destructor is a special method that runs when the garbage collector reclaims an object's memory. Unlike constructors, which initialize new objects, destructors perform cleanup just before an object is destroyed. Use of finalizers is significant when unmanaged resources need deterministic cleanup. Finalizers incur overhead and timing is nondeterministic—thus they should be used sparingly, typically paired with the ID disposable pattern for reliable resource release.

Code

```
using System;

namespace PNC
{
    public class Student
    {
        public string Name;

        public string Gender;

        public int RollNo;

        public Student(string name, string gender, int rollNo)
        {
            Name = name;

            Gender = gender;

            RollNo = rollNo;

            Console.WriteLine("Constructor: Student object created.");
        }

        public void Display()
        {
            Console.WriteLine("Student Details:");
        }
    }
}
```

```

        Console.WriteLine("Name: " + Name);

        Console.WriteLine("Gender: " + Gender);

        Console.WriteLine("Roll No: " + RollNo);
    }

    ~Student()
    {
        Console.WriteLine("Destructor: Student object destroyed.");
    }
}

class Program
{
    static void Main(string[] args)
    {
        CreateStudent();

        GC.Collect();

        GC.WaitForPendingFinalizers();

        Console.ReadLine();
    }

    static void CreateStudent()
    {
        PNC.Student student1 = new PNC.Student("Deep", "Male", 004);

        student1.Display();
    }
}

```

Output

```
Constructor: Student object created.  
Student Details:  
Name: Deep  
Gender: Male  
Roll No: 4  
Destructor: Student object destroyed.  
█
```

Conclusion

The Student class was successfully created inside the PNC namespace. The program correctly initialized the fields using a constructor and showed how a destructor can be used to release resources or display a message when the object is collected by the garbage collector.

Lab 12

Objective: To Create a Book class using private fields and public properties, and resolve naming conflicts in the constructor with the this keyword

Theory

Private Fields are variables declared inside a class and cannot be accessed directly from outside the class. Public Properties provide controlled access to private fields using get and set. The “this” keyword refers to the current instance of the class and is used to resolve naming conflicts between fields and constructor parameters.

Code

```
using System;

namespace BookLibrary
{
    public class Book
    {
        private string title;

        private string author;

        private double price;

        public string Title
        {
            get { return title; }

            set { title = value; }
        }

        public string Author
        {
            get { return author; }

            set { author = value; }
        }

        public double Price
```

```
{

    get { return price; }

    set { price = value; }

}

public Book(string title, string author, double price)

{

    this.title = title;

    this.author = author;

    this.price = price;

}

public void Display()

{

    Console.WriteLine("Book Information:");

    Console.WriteLine("Title: " + Title);

    Console.WriteLine("Author: " + Author);

    Console.WriteLine("Price: Rs." + Price);

}

}

class Program

{

    static void Main(string[] args)

    { Book myBook = new Book("DotNet", "Deep", 599.99);

        myBook.Display();

        Console.ReadLine();

    }

}

}
```

Output

```
Book Information:  
Title: DotNet  
Author: Deep  
Price: Rs.599.99
```

Conclusion

The 'Book' class was successfully implemented using private fields and public properties. The 'this' keyword effectively resolved naming conflicts in the constructor. This lab demonstrates encapsulation and clean object initialization.

Lab 13

Objective: To Implement a Rectangle class that overloads constructors to allow various ways to initialize length and width

Theory

Constructor Overloading allows a class to have multiple constructors with different parameter lists. It provides flexibility in object initialization depending on available data. A class in C# can define multiple constructors as long as their signatures are different.

Code

```
using system;

namespace Shapes
{
    public class Rectangle
    {
        public double Length { get; set; }

        public double Width { get; set; }

        public Rectangle(){
            Length = 4;
            Width = 4;}

        public Rectangle(double side){
            Length = side;
            Width = side;
        }

        public Rectangle(double length, double width){
            Length = length;
            Width = width;
        }

        public double Area() => Length * Width;
```



```
public double Perimeter() => 2 * (Length + Width);}

}

class Program
{
    static void Main()
    {
        var defaultRect = new Shapes.Rectangle();

        var square = new Shapes.Rectangle(4);

        var customRect = new Shapes.Rectangle(2, 4);

        Console.WriteLine($"Default: {defaultRect.Area()} area, {defaultRect.Perimeter()} perimeter");

        Console.WriteLine($"Square: {square.Area()} area, {square.Perimeter()} perimeter");

        Console.WriteLine($"Custom: {customRect.Area()} area, {customRect.Perimeter()} perimeter");

    }
}
```

Output

```
Default: 16 area, 16 perimeter
Square: 16 area, 16 perimeter
Custom: 8 area, 12 perimeter
```

Conclusion

This experiment successfully demonstrated constructor overloading by implementing a Rectangle class that provided multiple ways to initialize the length and width properties. The use of different constructor versions—default, single parameter, and two parameters illustrated how constructor overloading enhances the flexibility and usability of a class.

Lab 14

Objective: To Create a Complex Number class and overload the + operator to add two complex numbers

Theory

Operator Overloading allows user-defined types to define the behavior of standard operators. A complex number has a real part and an imaginary part, typically represented as $a + bi$. The + operator for complex numbers should add real parts and imaginary parts separately.

Code

```
using System;

namespace MathLibrary
{
    public class Complex {
        private double real;

        private double imaginary;

        public Complex(double real, double imaginary)
        {
            this.real = real;

            this.imaginary = imaginary;
        }

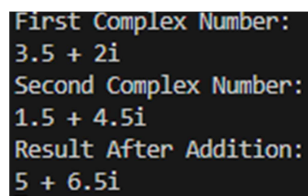
        public static Complex operator +(Complex c1, Complex c2)
        {
            return new Complex(c1.real + c2.real, c1.imaginary + c2.imaginary);
        }

        public void Display(){

            Console.WriteLine($"{real} + {imaginary}i");
        }
    }
}
```

```
}  
  
class Program{  
  
    static void Main(string[] args)  
  
    {  
  
        Complex num1 = new Complex(3.5, 2.0);  
  
        Complex num2 = new Complex(1.5, 4.5);  
  
        Complex result = num1 + num2;  
  
        Console.WriteLine("First Complex Number:");  
  
        num1.Display();  
  
        Console.WriteLine("Second Complex Number:");  
  
        num2.Display();  
  
        Console.WriteLine("Result After Addition:");  
  
        result.Display();  
  
        Console.ReadLine(); }  
  
}
```

Output

A screenshot of a terminal window showing the output of the program. The text is as follows:
First Complex Number:
3.5 + 2i
Second Complex Number:
1.5 + 4.5i
Result After Addition:
5 + 6.5i
The text is displayed in a monospaced font with a dark background and light-colored text.

```
First Complex Number:  
3.5 + 2i  
Second Complex Number:  
1.5 + 4.5i  
Result After Addition:  
5 + 6.5i
```

Conclusion

The Complex class was successfully implemented with overloaded + operator to add two complex numbers. The program correctly performs complex number addition using intuitive syntax.

Lab 15

Objective: To demonstrate inheritance by creating a **Vehicle** base class and a **Car** subclass that overrides a method

Theory

Inheritance in object-oriented programming allows a class to inherit members from another base class. Method Overriding enables a subclass to provide a specific implementation of a method that is already defined in its base class. In C#, overriding is done using the `virtual` keyword in the base class and the `override` keyword in the subclass.

Code

```
using System;

namespace TransportSystem

{
    public class Vehicle
    {
        public string Brand;

        public Vehicle(string brand)
        {
            Brand = brand;
        }

        public virtual void DisplayInfo()
        {
            Console.WriteLine("This is a vehicle.");
        }
    }

    public class Car : Vehicle
    {
        public string Model;
```

```
public Car(string brand, string model) : base(brand)
{
    Model = model;
}

// Overriding the base class method
public override void DisplayInfo(){
    Console.WriteLine("Car Brand: " + Brand);
    Console.WriteLine("Car Model: " + Model);
}
}

class Program{
    static void Main(string[] args)
    {
        Car myCar = new Car("Toyota", "Corolla");
        myCar.DisplayInfo();
        Console.ReadLine();
    }
}
}
```

Output

```
Car Brand: Toyota
Car Model: Corolla
```

Conclusion

The program successfully demonstrated inheritance and method overriding. The Car class inherited the Vehicle class and provided a specific implementation for the DisplayInfo() method.

Lab 16

Objective: To Define an abstract base class Shape containing an abstract method CalculateArea(), then implement concrete subclasses Circle and Square that provide their own area calculations

Theory

Abstraction is one of the key principles of object-oriented programming. It hides implementation details and shows only essential features. An abstract class cannot be instantiated and may contain abstract methods that must be overridden in derived classes. An abstract method has no body and is meant to be overridden by subclasses.

Code

```
using System;

namespace Geometry{

public abstract class Shape

{

    public abstract double CalculateArea();}

    public class Circle : Shape

    {

        public double Radius;

        public Circle(double radius)

        {

            Radius = radius;

        }

        public override double CalculateArea()

        {

            return Math.PI * Radius * Radius;

        }

    }

}
```

```
public class Square : Shape
{
    public double Side;

    public Square(double side)
    { Side = side; }

    public override double CalculateArea()
    {
        return Side * Side;
    }
}

class Program{

    static void Main(string[] args){

        Shape circle = new Circle(5);

        Shape square = new Square(4);

        Console.WriteLine("Area of Circle: " + circle.CalculateArea());

        Console.WriteLine("Area of Square: " + square.CalculateArea());

    }

}
```

Output

```
Area of Circle: 78.53981633974483
Area of Square: 16
```

Conclusion

The abstract class Shape was successfully created with an abstract method CalculateArea(). Concrete classes Circle and Square implemented their own logic to calculate the area, demonstrating abstraction and method overriding.

Lab 17

Objective: To Modify the Car class so that it uses the base keyword to invoke the StartEngine() method from its parent Vehicle class before executing its own override logic

Theory

In inheritance, the base keyword is used to access members of the base class from within a derived class. When overriding a method the base class's version of that method can be called using base.MethodName(). This allows extending the functionality of the base class rather than completely replacing it.

Code

```
using System;

namespace InheritanceDemo
{
    public class Vehicle
    {
        public virtual void StartEngine()
        {
            Console.WriteLine("Vehicle engine started.");
        }
    }

    public class Car : Vehicle
    {
        public override void StartEngine()
        {
            base.StartEngine();

            Console.WriteLine("Car engine is now running smoothly.");
        }
    }
}
```



```
}  
  
class Program  
  
{  
  
    static void Main(string[] args)  
  
    {  
  
        Car myCar = new Car();  
  
        myCar.StartEngine();  
  
    }  
  
}  
  
}
```

Output

```
Vehicle engine started.  
Car engine is now running smoothly.
```

Conclusion

This experiment successfully demonstrated the use of the `base` keyword in C# to invoke a method from a base class (Vehicle) within a derived class (Car). By modifying the Car class to call the `StartEngine()` method from its parent class using the `base` keyword, we were able to extend the functionality of the base method rather than completely overriding it.

Lab 18

Objective: To Demonstrate the ability of the object type to hold values of various data types, and use GetType() to display each value's actual runtime type.

Theory

In .NET, the GetType() method returns the actual runtime type of an object. This method is useful for determining the type of an object at runtime, which is essential for dynamic programming and polymorphism.

Code

```
using System;

namespace ObjectTypeDemo{

    class Program {

        static void Main(string[] args)

        {

            object number = 42;

            object pi = 3.14;

            object name = "Deep thapa";

            object flag = true;

            object date = DateTime.Now;

            Console.WriteLine($"Value: {number}, Type: {number.GetType()}");

            Console.WriteLine($"Value: {pi}, Type: {pi.GetType()}");

            Console.WriteLine($"Value: {name}, Type: {name.GetType()}");

            Console.WriteLine($"Value: {flag}, Type: {flag.GetType()}");

            Console.WriteLine($"Value: {date}, Type: {date.GetType()}");

        }

    }

}
```

Output

```
Value: 42, Type: System.Int32  
Value: 3.14, Type: System.Double  
Value: Deep thapa, Type: System.String  
Value: True, Type: System.Boolean  
Value: 14-Jun-25 10:17:55 AM, Type: System.DateTime
```

Conclusion

The object type was successfully used to store values of various data types. The GetType() method correctly identified each value's actual runtime type, proving that object supports runtime polymorphism.

Lab 19

Objective: To define a struct named Point containing x and y coordinates, using public, private, and internal access modifiers to illustrate encapsulation levels.

Theory

A struct in C# is a value type used to store small groups of related variables. Encapsulation is the concept of hiding internal object details and exposing only what's necessary. Access modifiers in C# define the scope and visibility of a variable or method.

Code

```
using System;

namespace Geometry
{
    public struct Point
    {
        public int X;

        private int Y;

        internal int Z;

        public Point(int x, int y, int z)
        {
            X = x;

            Y = y;

            Z = z;
        }

        public void Display()
        {
            Console.WriteLine($"X: {X}, Y: {Y}, Z: {Z}");
        }
    }
}
```

```

    public int GetY()

    {

        return Y;

    }

    public void SetY(int y){

        Y = y;}

}

class Program {

    static void Main(string[] args){

        Point p = new Point(10, 20, 30);

        Console.WriteLine("Accessing Public Field X: " + p.X);

        Console.WriteLine("Accessing Internal Field Z: " + p.Z);

        Console.WriteLine("Accessing Private Field Y via GetY(): " + p.GetY());

        p.SetY(50);

        Console.WriteLine("Modified Private Field Y via SetY(50): " + p.GetY());

        p.Display(); }

}

```

Output

```

Accessing Public Field X: 10
Accessing Internal Field Z: 30
Accessing Private Field Y via GetY(): 20
Modified Private Field Y via SetY(50): 50
X: 10, Y: 50, Z: 30

```

Conclusion

The Point struct successfully demonstrated the use of public, private, and internal access modifiers. Each level of encapsulation was illustrated through field visibility and controlled access.

Lab 20

Objective: To Develop a C# console application to manage a BCA Student database

Theory

The BCASStudentApp is a simple console application designed to manage student data using SQL Server as a backend database. This program allows for creating a database, adding student information, and performing basic CRUD operations such as viewing, updating, and filtering student records. The program follows the CRUD pattern, although Delete operations are not included in this specific example.

Code

```
using System;

using System.Collections.Generic;

namespace BCASStudentDatabase
{
    class Student {

        public int RollNo { get; set; }

        public string Name { get; set; }

        public string Gender { get; set; }

        public string Semester { get; set; }

        public void Display()

        {

            Console.WriteLine($"Roll No: {RollNo}, Name: {Name}, Gender: {Gender}, Semester: {Semester}");

        }

    }

    class Program {

        static List<Student> students = new List<Student>();

        static void Main(string[] args)

        {
```

```
int choice;

do {

    Console.Clear();

    Console.WriteLine("==== BCA Student Database =====");

    Console.WriteLine("1. Add Student");

    Console.WriteLine("2. Display All Students");

    Console.WriteLine("3. Search by Roll No");

    Console.WriteLine("4. Exit");

    Console.Write("Enter your choice: ");

    choice = Convert.ToInt32(Console.ReadLine());

    switch (choice)
    {

        case 1:

            AddStudent();

            break;

        case 2:

            DisplayAllStudents();

            break;

        case 3:

            SearchStudent();

            break;

        case 4:

            Console.WriteLine("Exiting...");

            break;

        default:

            Console.WriteLine("Invalid choice!");

            break;

    }

}
```

```

    }

    Console.WriteLine("\nPress any key to continue...");

    Console.ReadKey();

} while (choice != 4);
}

static void AddStudent()
{
    Student s = new Student();

    Console.Write("Enter Roll No: ");

    s.RollNo = Convert.ToInt32(Console.ReadLine());

    Console.Write("Enter Name: ");

    s.Name = Console.ReadLine();

    Console.Write("Enter Gender: ");

    s.Gender = Console.ReadLine();

    Console.Write("Enter Semester: ");

    s.Semester = Console.ReadLine();

    students.Add(s);

    Console.WriteLine("Student added successfully.");
}

static void DisplayAllStudents()
{
    if (students.Count == 0)
    {
        Console.WriteLine("No student records found.");

        return;
    }

    Console.WriteLine("\nList of Students:");

```



```
        foreach (Student s in students)

        {

            s.Display();

        }

    }

    static void SearchStudent()

    {

        Console.Write("Enter Roll No to search: ");

        int roll = Convert.ToInt32(Console.ReadLine());

        bool found = false;

        foreach (Student s in students)

        {

            if (s.RollNo == roll)

            {

                Console.WriteLine("Student Found:");

                s.Display();

                found = true;

                break;

            }

        }

        if (!found)

        {

            Console.WriteLine("Student not found.");

        }

    }

}
```

Output

```
==== BCA Student Database ====
1. Add Student
2. Display All Students
3. Search by Roll No
4. Exit
Enter your choice: 1
Enter Roll No: 4
Enter Name: Deep
Enter Gender: M
Enter Semester: 5
Student added successfully.

Press any key to continue...
```

```
Enter your choice: 2

List of Students:
Roll No: 4, Name: Deep, Gender: M, Semester: 5
Roll No: 5, Name: Ayush, Gender: M, Semester: 4
```

```
Enter your choice: 3
Enter Roll No to search: 4
Student Found:
Roll No: 4, Name: Deep, Gender: M, Semester: 5
```

Conclusion

The C# console application was successfully developed to manage a simple BCA Student database using object-oriented principles. It allows adding, displaying, and searching student records dynamically in-memory.